Task 1 (AWS Cafe Challenge Portfolio)

Challenge 4 - Creating a VPC Networking Environment for the Café

Academic Year	Module	Assessment Number	Assessment Type
2026	5CS048 - Cloud Systems	Task 1 Challenge 4	Portfolio

Student ID: 2462250

Student Name: Praju Maharjan

Group: L5CG9

Module Leader: Mr. Deepson Shrestha

Tutor: Mr. Prabin Sapkota

Submitted On: 23 April 2026

Table of Contents

Challenge (Cafe) Lab: Creating a VPC Networking Environment for the Cafe

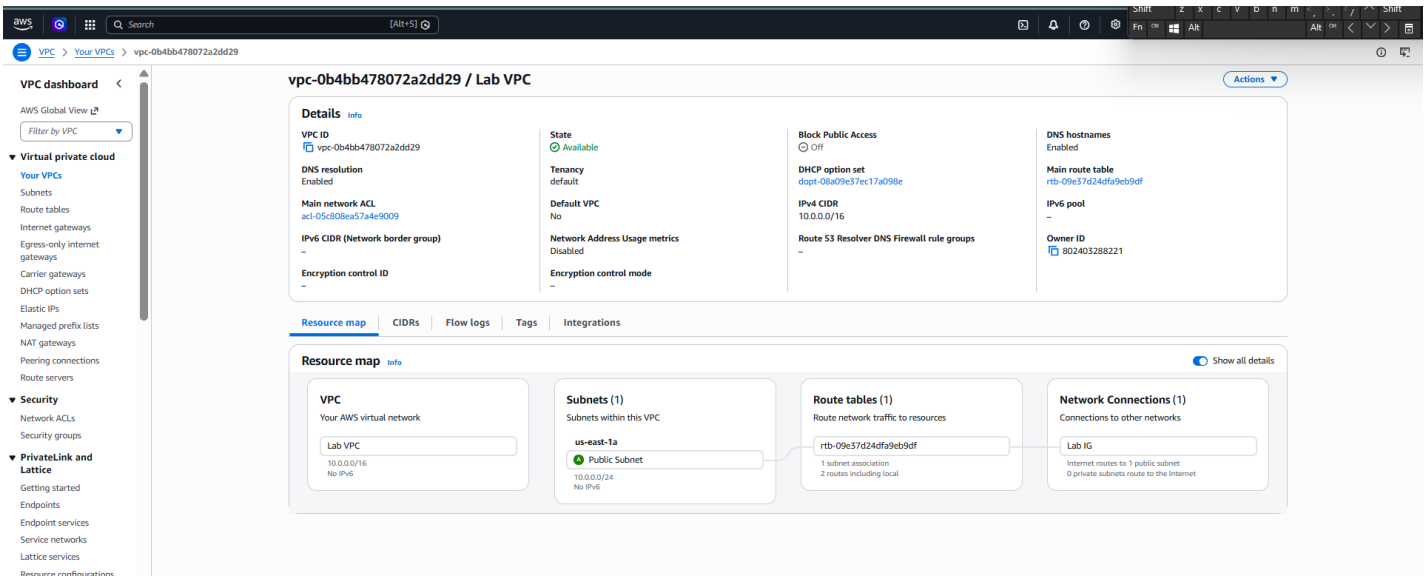
Challenge 1: Creating a VPC network for secure administration of the cafe application server

---

Task 1: Creating a public subnet and internet gateway

Show a screenshot of the Amazon VPC console displaying:

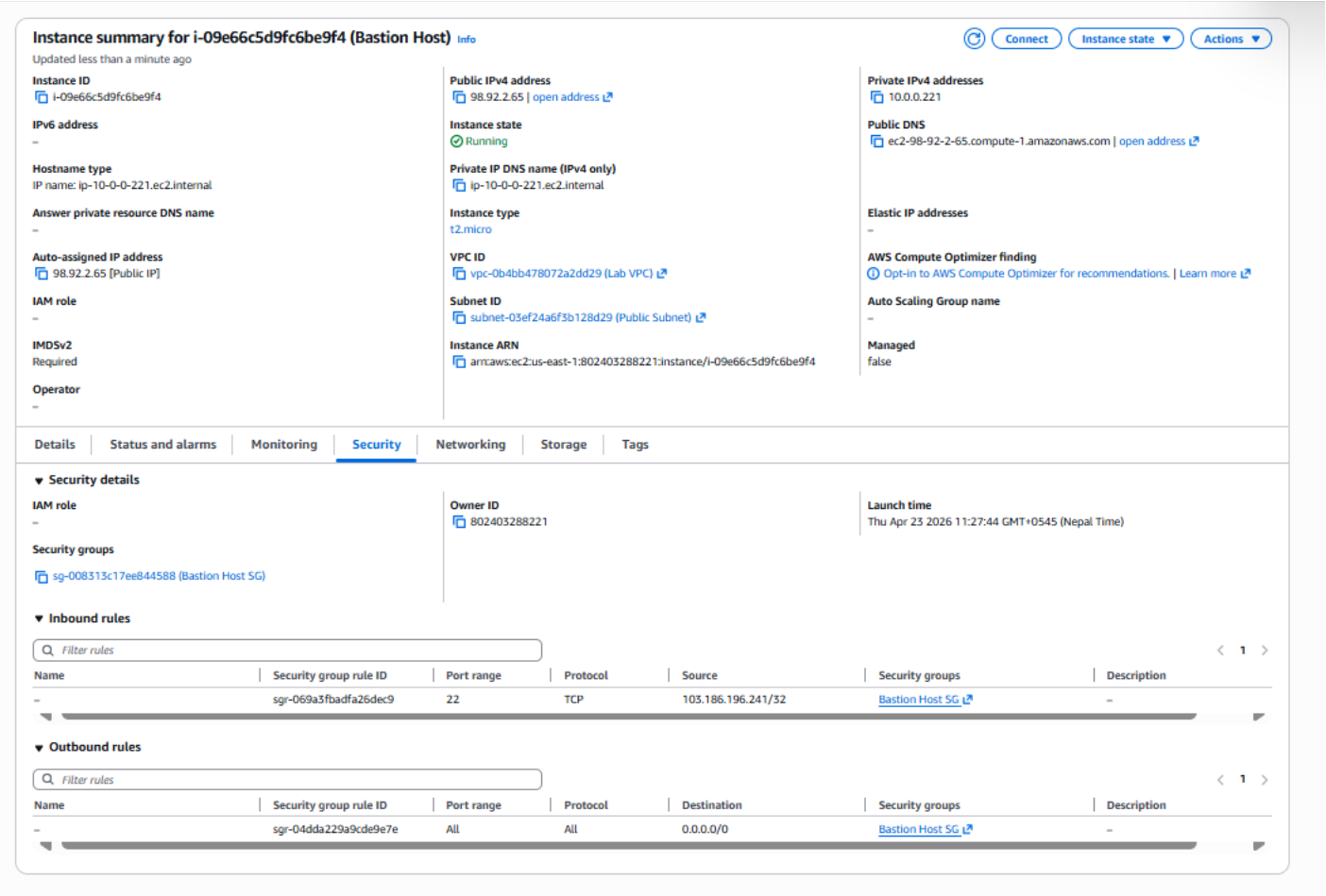
- The Public Subnet (10.0.0.0/24 in Availability Zone a)
- The Internet Gateway attached to the Lab VPC
- The route table associated with the Public Subnet showing the 0.0.0.0/0 route pointing to the Internet Gateway



Task 2: Creating the bastion host

Show a screenshot of the Amazon EC2 console with the Bastion Host instance details:

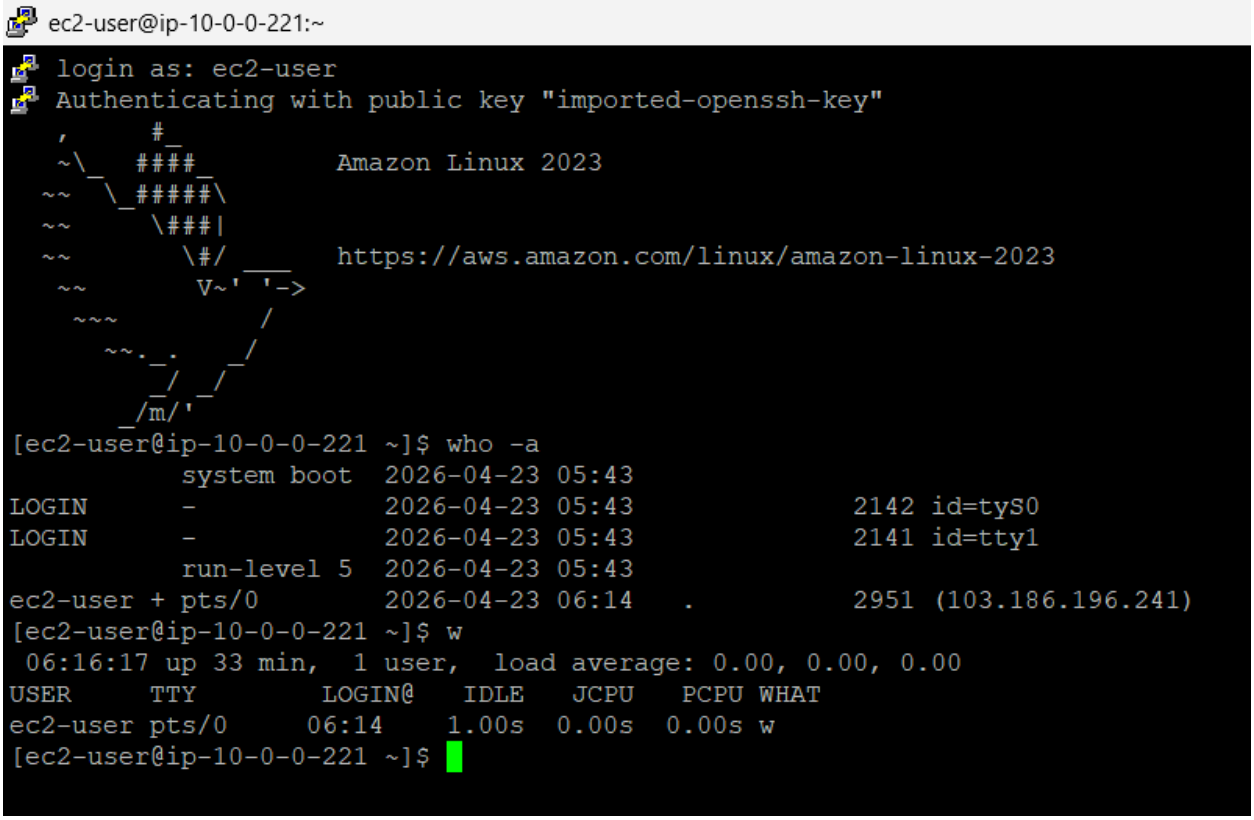
- **Status: Running**
- **Subnet: Public Subnet**
- **Security Group: Bastion Host SG (inbound SSH port 22 from My IP only)**



Task 3: Testing connection to the bastion host

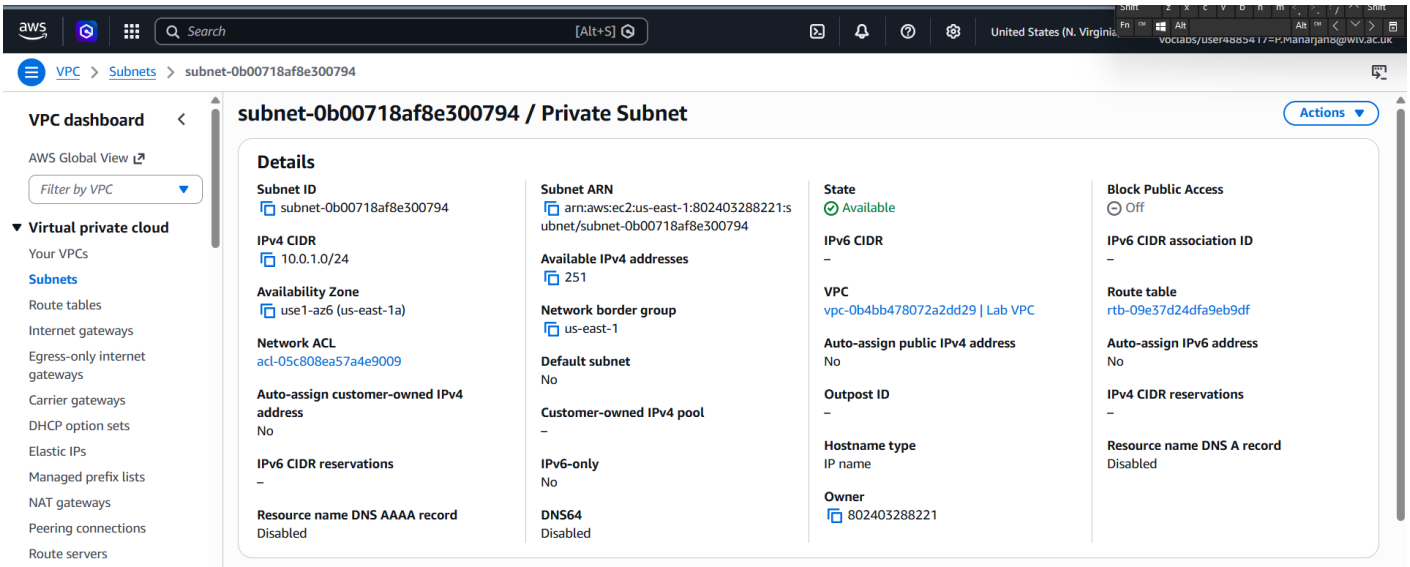
Show a screenshot of your local terminal or SSH session connected to the Bastion

Host. The screenshot must clearly show successful login to the bastion host.



Task 4: Creating a private subnet

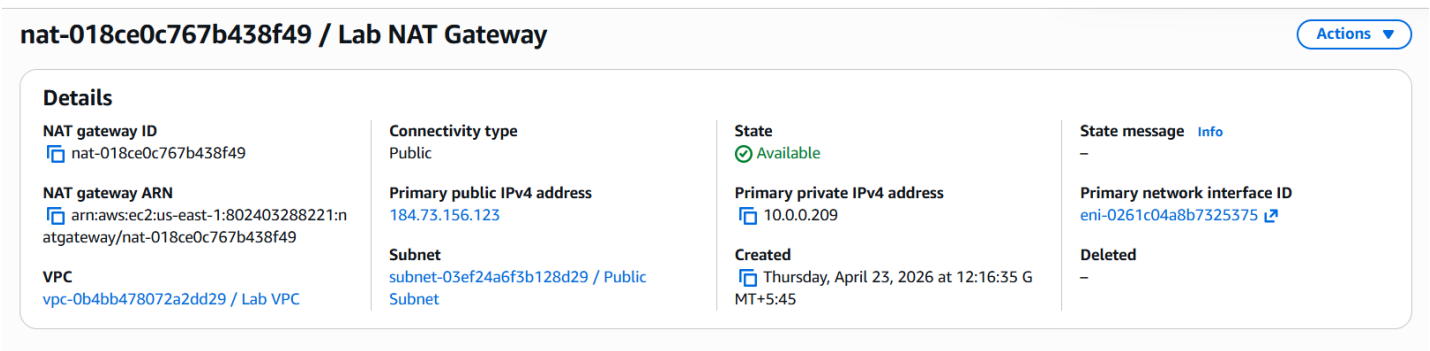
Show a screenshot of the Amazon VPC console displaying the Private Subnet (10.0.1.0/24 in the same Availability Zone as the Public Subnet).



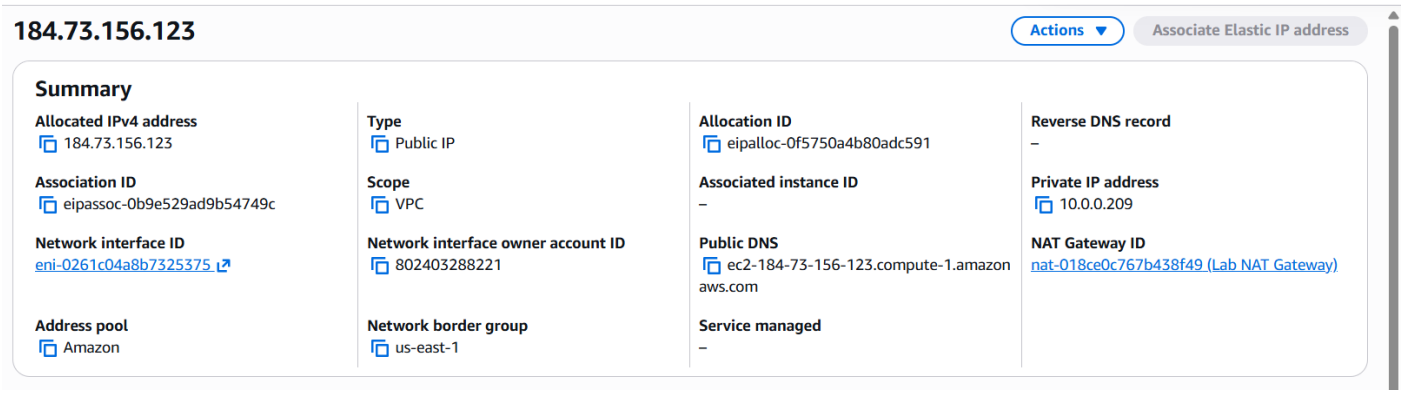
Task 5: Creating a NAT gateway

Show a screenshot of the Amazon VPC console with:

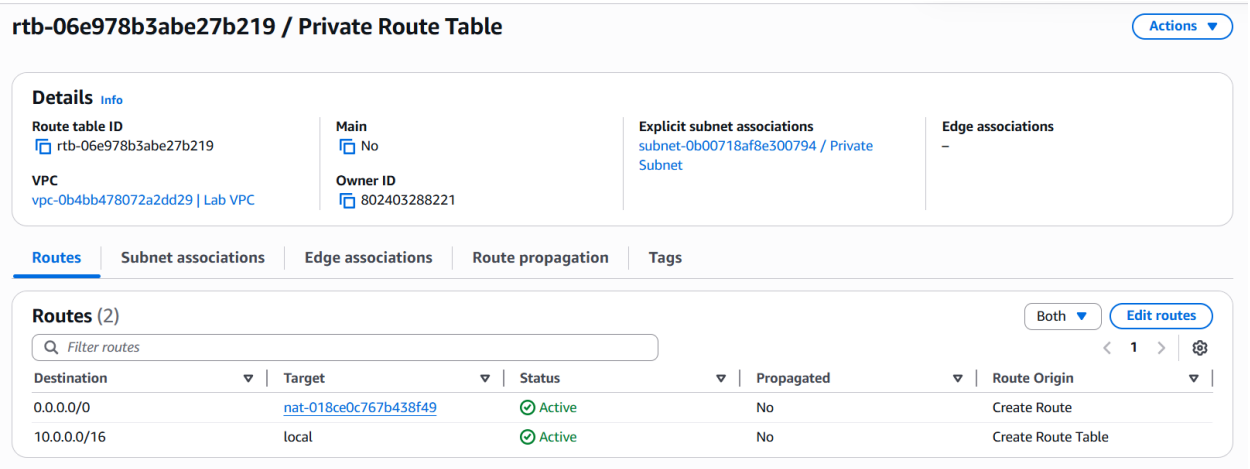
- **The Lab NAT Gateway created in the Public Subnet**



- **Its associated Elastic IP address**



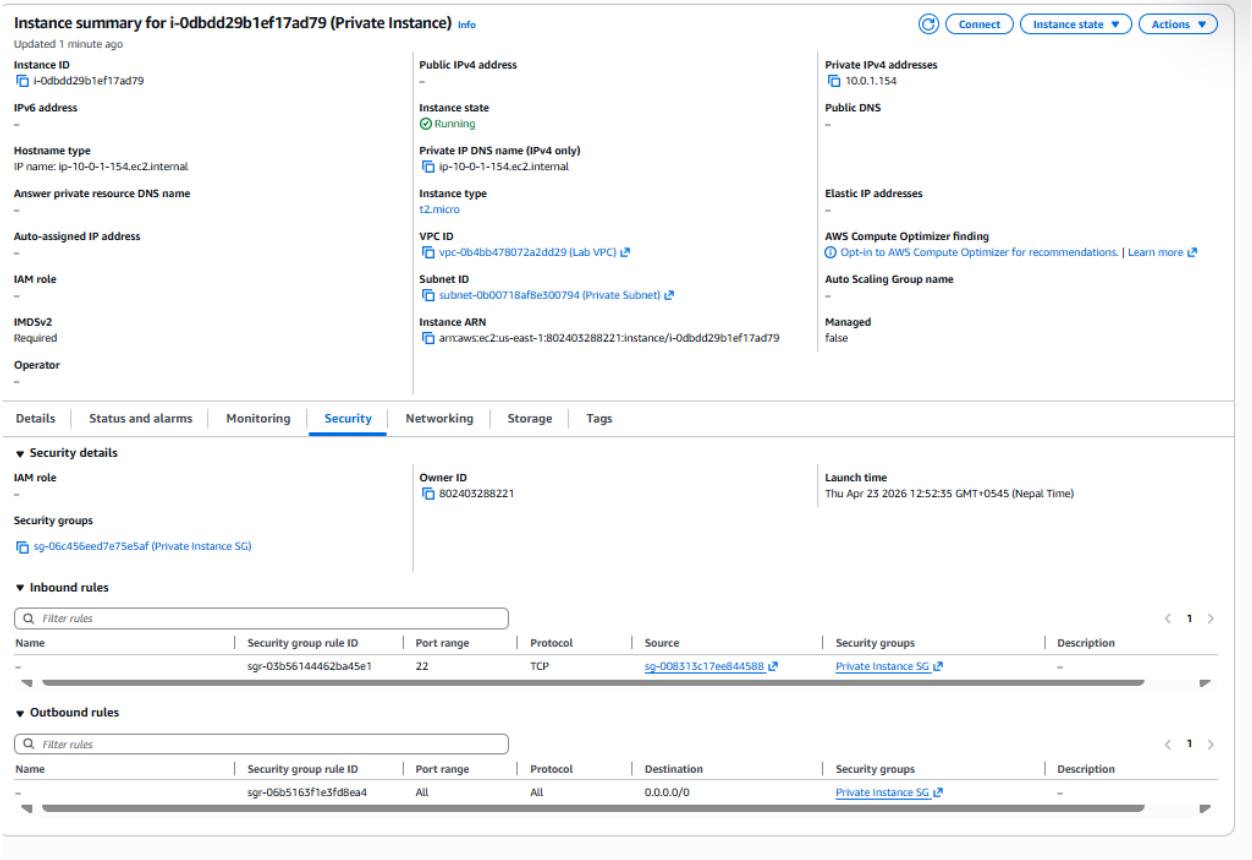
• The Private Route Table showing the 0.0.0.0/0 route pointing to the NAT Gateway and attached to the Private Subnet



Task 6: Creating the private EC2 instance

Show a screenshot of the Amazon EC2 console with the Private Instance details:

- **Status: Running**
- **Subnet: Private Subnet**
- **Security Group: Private Instance SG (inbound SSH port 22 sourced from the Bastion Host SG)**



Task 7 and 8: Testing SSH passthrough and connection from bastion to private instance

Show a screenshot of the terminal session showing:

- **Successful SSH connection from the Bastion Host to the Private Instance using SSH agent forwarding**
- **The output of the ping 8.8.8.8 command confirming the private instance has internet access**

```
[ec2-user@ip-10-0-0-221 ~]$ ssh ec2-user@10.0.1.154
The authenticity of host '10.0.1.154 (10.0.1.154)' can't be established.
ED25519 key fingerprint is SHA256:wQaCYNlaW0IvXqEZVDA0iY3u7CwPUjT/Zmp5hGx0uvk.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.0.1.154' (ED25519) to the list of known hosts.
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
[ec2-user@ip-10-0-1-154 ~]$ ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=116 time=2.16 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=116 time=1.67 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=116 time=1.71 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=116 time=1.71 ms
64 bytes from 8.8.8.8: icmp_seq=5 ttl=116 time=1.77 ms
64 bytes from 8.8.8.8: icmp_seq=6 ttl=116 time=1.70 ms
64 bytes from 8.8.8.8: icmp_seq=7 ttl=116 time=1.72 ms
64 bytes from 8.8.8.8: icmp_seq=8 ttl=116 time=1.74 ms
64 bytes from 8.8.8.8: icmp_seq=9 ttl=116 time=1.71 ms
64 bytes from 8.8.8.8: icmp_seq=10 ttl=116 time=1.68 ms
64 bytes from 8.8.8.8: icmp_seq=11 ttl=116 time=6.52 ms
64 bytes from 8.8.8.8: icmp_seq=12 ttl=116 time=1.67 ms
64 bytes from 8.8.8.8: icmp_seq=13 ttl=116 time=1.74 ms
64 bytes from 8.8.8.8: icmp_seq=14 ttl=116 time=1.71 ms
64 bytes from 8.8.8.8: icmp_seq=15 ttl=116 time=1.65 ms
64 bytes from 8.8.8.8: icmp_seq=16 ttl=116 time=1.68 ms
^C
--- 8.8.8.8 ping statistics ---
16 packets transmitted, 16 received, 0% packet loss, time 15028ms
rtt min/avg/max/mdev = 1.650/2.033/6.521/1.164 ms
[ec2-user@ip-10-0-1-154 ~]$
```

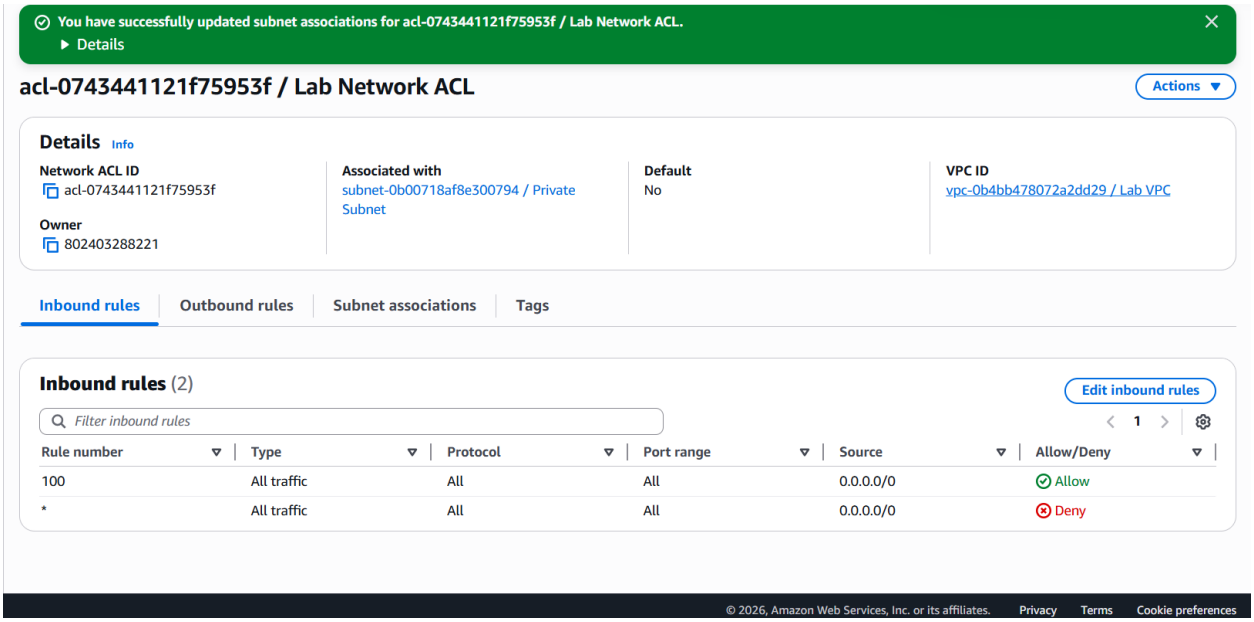
Challenge 2: Enhancing the security layer using Network ACLs

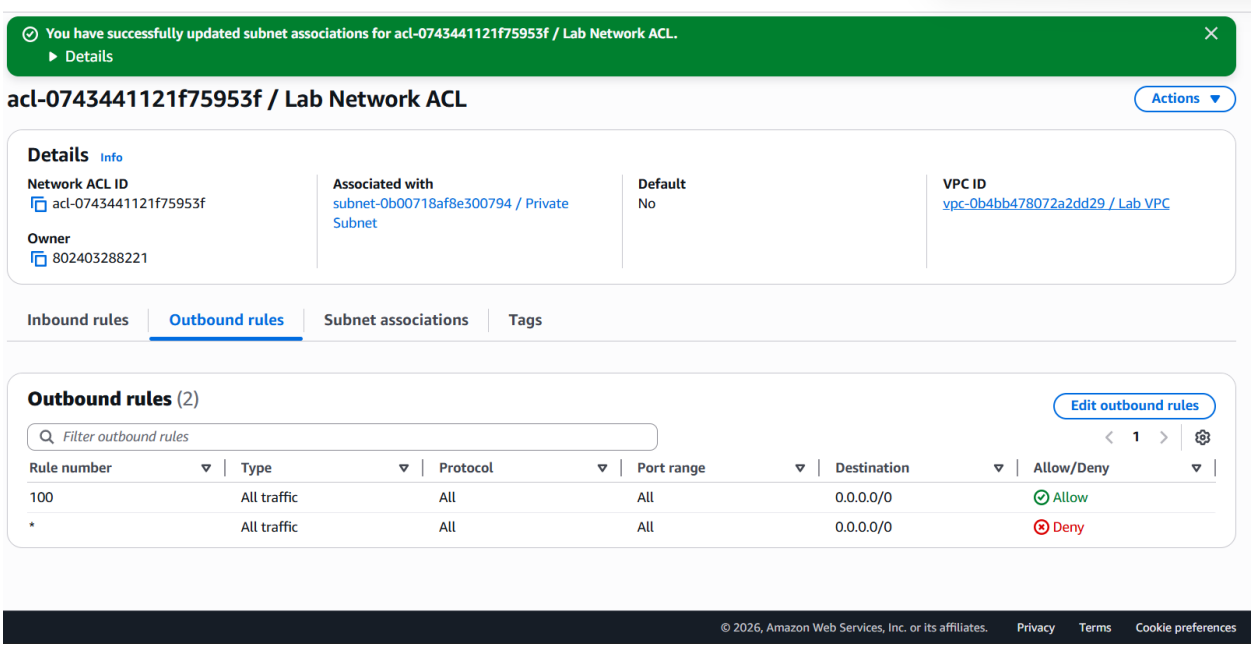
Task 9: Creating and configuring a custom network ACL

Show a screenshot of the Amazon VPC console displaying the Lab Network ACL with:

- **Its inbound and outbound rules configured to allow all traffic**

- **The ACL associated with the Private Subnet**





Task 10: Testing the custom network ACL

Show two screenshots:

- **The terminal from the Private Instance showing the ping command to the Test Instance private IP address before applying the deny rule (ping should be successful)**

```
[ec2-user@ip-10-0-1-154 ~]$ ping 10.0.0.93
PING 10.0.0.93 (10.0.0.93) 56(84) bytes of data.
64 bytes from 10.0.0.93: icmp_seq=1 ttl=127 time=1.41 ms
64 bytes from 10.0.0.93: icmp_seq=2 ttl=127 time=0.429 ms
64 bytes from 10.0.0.93: icmp_seq=3 ttl=127 time=0.498 ms
64 bytes from 10.0.0.93: icmp_seq=4 ttl=127 time=0.450 ms
64 bytes from 10.0.0.93: icmp_seq=5 ttl=127 time=0.590 ms
64 bytes from 10.0.0.93: icmp_seq=6 ttl=127 time=0.749 ms
64 bytes from 10.0.0.93: icmp_seq=7 ttl=127 time=0.475 ms
64 bytes from 10.0.0.93: icmp_seq=8 ttl=127 time=0.438 ms
64 bytes from 10.0.0.93: icmp_seq=9 ttl=127 time=0.542 ms
64 bytes from 10.0.0.93: icmp_seq=10 ttl=127 time=0.472 ms
64 bytes from 10.0.0.93: icmp_seq=11 ttl=127 time=0.484 ms
64 bytes from 10.0.0.93: icmp_seq=12 ttl=127 time=0.420 ms
64 bytes from 10.0.0.93: icmp_seq=13 ttl=127 time=0.464 ms
64 bytes from 10.0.0.93: icmp_seq=14 ttl=127 time=0.454 ms
64 bytes from 10.0.0.93: icmp_seq=15 ttl=127 time=0.461 ms
64 bytes from 10.0.0.93: icmp_seq=16 ttl=127 time=0.457 ms
64 bytes from 10.0.0.93: icmp_seq=17 ttl=127 time=0.471 ms
64 bytes from 10.0.0.93: icmp_seq=18 ttl=127 time=0.457 ms
64 bytes from 10.0.0.93: icmp_seq=19 ttl=127 time=1.31 ms
64 bytes from 10.0.0.93: icmp_seq=20 ttl=127 time=0.475 ms
64 bytes from 10.0.0.93: icmp_seq=21 ttl=127 time=0.418 ms
64 bytes from 10.0.0.93: icmp_seq=22 ttl=127 time=0.478 ms
64 bytes from 10.0.0.93: icmp_seq=23 ttl=127 time=0.469 ms
64 bytes from 10.0.0.93: icmp_seq=24 ttl=127 time=0.846 ms
64 bytes from 10.0.0.93: icmp_seq=25 ttl=127 time=0.447 ms
64 bytes from 10.0.0.93: icmp_seq=26 ttl=127 time=0.441 ms
64 bytes from 10.0.0.93: icmp_seq=27 ttl=127 time=0.464 ms
64 bytes from 10.0.0.93: icmp_seq=28 ttl=127 time=0.420 ms
64 bytes from 10.0.0.93: icmp_seq=29 ttl=127 time=0.415 ms
64 bytes from 10.0.0.93: icmp_seq=30 ttl=127 time=0.434 ms
64 bytes from 10.0.0.93: icmp_seq=31 ttl=127 time=0.479 ms
64 bytes from 10.0.0.93: icmp_seq=32 ttl=127 time=0.622 ms
64 bytes from 10.0.0.93: icmp_seq=33 ttl=127 time=0.494 ms
64 bytes from 10.0.0.93: icmp_seq=34 ttl=127 time=0.603 ms
```

- **The same terminal after modifying the Network ACL to deny ICMP traffic showing that the ping no longer receives replies**

```
[ec2-user@ip-10-0-1-154 ~]$ ping 10.0.0.93
PING 10.0.0.93 (10.0.0.93) 56(84) bytes of data.
^C
--- 10.0.0.93 ping statistics ---
24 packets transmitted, 0 received, 100% packet loss, time 23884ms
```

Description

Write in your own wording and clearly cover all of the following (about 250 to 450 words):

1. what you built and what you did in that lab
2. one security control you used and why it matters
3. one performance or reliability feature that is present and how it helps
4. one issue you had and how you fixed it (or one likely issue and how you would fix it)

Ans:

In this lab, I built a secure, multi-layer VPC networking environment on AWS to support the café's web application infrastructure. The work was divided into two main challenges. In Challenge 1, I created a custom VPC (Lab VPC) containing a Public Subnet (10.0.0.0/24) and a Private Subnet (10.0.1.0/24), both placed in the same Availability Zone. I then attached an Internet Gateway to the VPC and updated the public route table to direct outbound traffic (0.0.0.0/0) through it. A Bastion Host EC2 instance was launched in the Public Subnet with a security group that only permits SSH access on port 22 from my own IP address, acting as a controlled entry point into the network. A NAT Gateway was placed in the Public Subnet with an associated Elastic IP address, and a Private Route Table was created to route internet-bound traffic from the Private Subnet through the NAT Gateway. Finally, a Private Instance EC2 was launched in the Private Subnet with a dedicated security group (Private Instance SG) that allows SSH only from the Bastion Host security group. I configured SSH agent forwarding to connect from the Bastion Host to the Private Instance without transferring private keys, and confirmed internet access from the private instance using the ping 8.8.8.8 command. In Challenge 2, I inspected the default Network ACL of the Lab VPC and created a custom Network ACL (Lab Network ACL) associated with the Private Subnet. I first configured the ACL to allow all inbound and outbound traffic, then launched a Test Instance in the Public Subnet and confirmed reachability via ping from the Private Instance. I then added a deny rule for ICMP traffic to the Test Instance's private IP address (/32) with a lower rule number so it was evaluated first, which successfully blocked all further ping responses.

One security control I used was the Bastion Host with a restricted Security Group. By only allowing inbound SSH from my own IP address and requiring all administrative access to the Private Instance to pass through this single, auditable entry point, the architecture removes any direct public exposure of the private EC2 instance. This matters because it significantly reduces the attack surface: even if

an attacker discovers the Private Instance's IP, there is no publicly reachable path to it. The combination of a Public-facing Bastion and a Private Instance SG that only trusts traffic originating from the Bastion Host security group enforces a layered, least-privilege access model across the entire environment.

A reliability feature present in this architecture is the NAT Gateway. Unlike a self-managed NAT instance running on an EC2 virtual machine, the AWS-managed NAT Gateway is highly available within its Availability Zone and scales automatically to handle varying amounts of traffic. This means that instances in the Private Subnet can reliably download security patches and software updates from the internet without any manual intervention or risk of the NAT component becoming a single point of failure under load. For a production workload, placing NAT Gateways in multiple Availability Zones would further increase resilience across the full region.

One issue I encountered was with SSH agent forwarding during Task 7. After adding both key pairs to ssh-agent using the ssh-add command, the initial SSH connection from the Bastion Host to the Private Instance was refused. The problem was that the Private Instance SG rule had been set with the source as a CIDR range rather than the Bastion Host security group ID, which meant the inbound SSH rule did not correctly recognise traffic arriving from the Bastion Host. I resolved this by editing the inbound rule of the Private Instance SG to reference the Bastion Host SG directly as its source, rather than an IP address range. This also serves as a best practice because it makes the security group rule dynamic: if the Bastion Host's IP address ever changes, the rule continues to work without any manual update.